

Hackathon Submission Guide

How to package your project so the testing pipeline can score it

This document defines **exactly how your submission must be structured** so that our automated testing pipeline can evaluate it. Submissions that do not follow this contract cannot be scored, so please read it carefully before the launch.

The rule of thumb: we should be able to clone your repo, drop in our own test data, run a single command, and get your predictions out — with no manual fixes from us.

1. How your submission gets tested

Here is the exact sequence our pipeline runs against every submission. Nothing else happens, and nothing is done by hand.

1. **Clone** your public GitHub repository from the URL you submit.
2. **Install** your dependencies (see Section 6).
3. **Replace** the contents of your `data/` folder with our held-out test data.
4. **Run** your shell command file. It must (a) generate the features your model needs from whatever is in `data/`, then (b) load your pickled model and produce predictions.
5. **Collect** the output file your run produced and score it.

If any of these steps fails, your submission gets a zero for that run, so **test the full sequence yourself on a clean machine** before you submit.

2. Required repository structure

Your repo **must** contain at least the following at its root:

```
<your-repo>/
├── run.sh           # Entry point – the one command we run (required)
├── requirements.txt # Your dependencies (required; see Section 6)
├── data/           # Input data folder – we overwrite this at test time
│   └── ...         # Your sample data goes here during development
├── pickle/         # Holds your trained model artifact
│   └── model.pkl   # Your pickled model (required)
├── src/            # Your code (name it however you like)
└── README.md       # Short description + how to run it locally
```

You can add anything else you need. The four required items are `run.sh`, `requirements.txt`, the `data/` folder, and a model file inside `pickle/`.

3. The shell command file (`run.sh`)

This is the single entry point we use. It holds the key arguments and orchestrates everything: feature generation and prediction.

Contract

`run.sh` must accept three positional arguments, and must also work with sensible defaults so you can run it locally with no arguments:

```
./run.sh <DATA_DIR> <MODEL_PATH> <OUTPUT_PATH>
```

Argument	What it is	Default we recommend
DATA_DIR	Folder containing the input data	./data
MODEL_PATH	Path to your pickled model	./pickle/model.pkl
OUTPUT_PATH	Where to write your predictions	./output/predictions.csv

At test time we will call it with our own paths, e.g.:

```
./run.sh ./data ./pickle/model.pkl ./output/predictions.csv
```

Example run.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Accept arguments, fall back to defaults for local runs
DATA_DIR="${1:-./data}"
MODEL_PATH="${2:-./pickle/model.pkl}"
OUTPUT_PATH="${3:-./output/predictions.csv}"

mkdir -p "$(dirname "$OUTPUT_PATH")"

# 1. Generate the features the model expects from the data
python src/generate_features.py \
  --data-dir "$DATA_DIR" \
  --out features.parquet

# 2. Load the pickled model and produce predictions
python src/predict.py \
  --features features.parquet \
  --model "$MODEL_PATH" \
  --output "$OUTPUT_PATH"

echo "Done. Predictions written to $OUTPUT_PATH"
```

Requirements for run.sh

- It must be at the **root** of the repo and named exactly `run.sh`.

- It must be **executable** (`chmod +x run.sh`) — but it should also run via `bash run.sh ...` just in case.
- It must run **end to end with one invocation** — feature generation and prediction together. Do not require us to run multiple scripts in order.
- It must **not** require interactive input (no prompts, no “press any key”).
- It should **fail loudly** (`set -euo pipefail`) rather than silently produce a bad output file.

4. The data/ folder

This is where input data lives. **At test time we delete your sample data and copy in our held-out test set with the same schema.**

What this means for you:

- Your code must read **whatever is inside data/ at run time** — do not hardcode specific filenames unless you are certain they will exist in our test set. Prefer reading by pattern, by the documented filename(s), or by iterating the folder.
- The **schema, column names, and file format** of our test data match the sample data described at the launch. We change the rows, not the structure.
- Do not assume the **size** of the data. Your pipeline should handle a different number of records than your sample.
- Keep your own sample data small and committed so the repo runs out of the box during your own testing.

5. The pickle/ directory and your model

- Your trained model must be saved as a pickle file inside the `pickle/` directory (e.g., `pickle/model.pkl`).
- The model must be **already trained and committed** — we do not retrain. The test run only generates features and predicts.
- Make sure the model **loads and runs with the exact library versions pinned in requirements.txt**. A model pickled with one version of scikit-learn / xgboost / etc. may not unpickle under another. This is the single most common reason submissions fail, so verify it on a clean environment.
- If the file is large, make sure it is actually committed (watch out for `.gitignore` rules or Git LFS that we cannot pull). The file must come down with a plain `git clone`.

6. Dependencies

We need to reproduce your environment automatically. Provide **one** of the following:

- A `requirements.txt` with **pinned versions** (`pandas==2.2.2`, not `pandas`). This is the default — we run `pip install -r requirements.txt` in a fresh Python environment.

- If you need system packages or a non-Python stack, include a `Dockerfile` and tell us in your README. (Confirm with the organizers whether Docker submissions are accepted before relying on this.)

Rules:

- **Pin every version.** Unpinned dependencies break unpickling and reproducibility.
- Your run must **not depend on internet access** during testing — download nothing at run time. Models, weights, and lookups must be in the repo.
- State your **Python version** in the README.

7. The output

- Your run must write predictions to the `OUTPUT_PATH` it is given (default `./output/predictions.csv`).
- The output **format and columns must match the format announced at the launch**. Producing the right answers in the wrong format scores zero.
- Write the file fresh on every run. Do not append to an existing one.

8. Reproducibility rules (please don't lose points here)

- **Set seeds** anywhere randomness affects predictions.
- **No absolute paths.** Use the arguments / relative paths only — `/Users/you/Desktop/...` will not exist on our machine.
- **No interactive or notebook-only steps.** Everything must run from `run.sh`.
- **No network calls** at run time.
- Test on a **clean clone in a fresh environment**, not your dev machine where everything is already installed.

9. Submission checklist

Before you submit, confirm every one of these:

- ☐ Repo is on GitHub and set to public.
- ☐ `run.sh` is at the root, executable, and runs end to end with one command.
- ☐ `run.sh` accepts `DATA_DIR`, `MODEL_PATH`, `OUTPUT_PATH` as arguments (with defaults).
- ☐ `data/` folder exists and your code reads from it dynamically.
- ☐ Trained model is committed under `pickle/` and loads cleanly.
- ☐ `requirements.txt` lists pinned versions of everything.
- ☐ Output is written to `OUTPUT_PATH` in the announced format.
- ☐ You ran the full sequence on a fresh clone and it produced a valid output file.

- ☐ No absolute paths, no prompts, no internet dependency at run time.

10. What you submit

At the launch / via the submission form, provide:

1. Your **public GitHub repository URL**.
2. The **exact command** you expect us to run (it should just be `./run.sh ./data ./pickle/model.pkl ./output/predictions.csv`, but tell us if anything differs).
3. Your **team name**, team members and college name.
4. **Email your submissions to sunitha.k@netelixir.us by 19th July, 2026 10:00 PM IST.**

Quick mental model

Clone → install → drop our data into data/ → ./run.sh → read predictions from output/

If that works on a machine that has never seen your project, you're good. If it needs a human to fix anything, it isn't ready.

Good luck — we're excited to see what you build.